

Absolutely. For an interview, you should know the flow, annotations, exception hierarchy, handler resolution, and how to return a proper API error response.

Spring Boot Exception Handling — Interview Notes

1. What is exception handling?

Exception handling is the mechanism used to handle runtime errors gracefully instead of allowing the application to return an uncontrolled error response.

Example:

Java

```
@GetMapping("/users/{id}")
public User getUser(@PathVariable Long id) {
    return userService.getUser(id);
}
```

Suppose the user doesn't exist:

Java

```
throw new UserNotFoundException("User not found");
```

Instead of returning a generic `500 Internal Server Error`, we can return:

JSON

```
{
  "status": 404,
  "message": "User not found"
}
```

2. Three important annotations

`@ExceptionHandler`

Used to define a method that handles a particular exception.

Java

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<String> handleUserNotFound(
    UserNotFoundException ex) {
```

```
        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}
```

It basically says:

"When this exception occurs, execute this method."

`@ControllerAdvice`

Makes a class a **global exception-handling component** for controllers.

Java

```
@ControllerAdvice
public class GlobalExceptionHandler {

    @ExceptionHandler(UserNotFoundException.class)
    public ResponseEntity<String> handleUserNotFound(
        UserNotFoundException ex) {

        return ResponseEntity
            .status(HttpStatus.NOT_FOUND)
            .body(ex.getMessage());
    }
}
```

Now the handler can handle exceptions thrown from multiple controllers.

`@RestControllerAdvice`

Combination of:

Java

```
@ControllerAdvice
@ResponseBody
```

For REST APIs, this is usually more convenient.

Java

```
@RestControllerAdvice
public class GlobalExceptionHandler {
```

3. How the exception flow works

This is the important interview concept.

Suppose:

```
Controller
  ↓
Service
  ↓
Exception thrown
  ↓
Spring MVC exception resolution
  ↓
ExceptionHandler matching
  ↓
Global Exception Handler
  ↓
HTTP response
```

For example:

Java

```
@GetMapping("/{id}")
public User getUser(@PathVariable Long id) {

    throw new UserNotFoundException("User not found");
}
```

Spring's exception-resolution mechanism looks for an appropriate `@ExceptionHandler`.

It does **not** mean that `@ControllerAdvice` itself "catches everything."

A better interview statement is:

"When an exception propagates out of a controller, Spring MVC's exception resolution mechanism searches for a matching `@ExceptionHandler` method, including handlers declared through `@ControllerAdvice`."

4. How does Spring choose the handler?

This is exactly what you asked about earlier.

Suppose we have:

Java

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<?> handleUserNotFound(...) {
}
```

and:

Java

```
@ExceptionHandler(Exception.class)
public ResponseEntity<?> handleGenericException(...) {
}
```

And:

Java

```
throw new UserNotFoundException();
```

Spring chooses:

Java

```
UserNotFoundException.class
```

because it is **more specific**.

Think of the hierarchy:

```
Throwable
  ↓
Exception
  ↓
RuntimeException
  ↓
UserNotFoundException
```

If there is no handler for `UserNotFoundException`, Spring can find a handler for its superclass.

For example:

Java

```
@ExceptionHandler(RuntimeException.class)
```

can handle it.

And finally:

Java

```
@ExceptionHandler(Exception.class)
```

can act as a broad fallback.

Interview answer

"Spring resolves the most specific matching exception handler. If an exact handler isn't available, it can match a handler for a superclass of the thrown exception."

5. Custom exception

Usually we create application-specific exceptions.

Java

```
public class UserNotFoundException extends RuntimeException {  
  
    public UserNotFoundException(String message) {  
        super(message);  
    }  
}
```

Then:

Java

```
throw new UserNotFoundException("User with ID 10 not found");
```

And handle it:

Java

```
@ExceptionHandler(UserNotFoundException.class)  
public ResponseEntity<?> handleUserNotFound(  
    UserNotFoundException ex) {  
  
    return ResponseEntity  
        .status(HttpStatus.NOT_FOUND)
```

```
.body(ex.getMessage());
```

```
}
```

6. Handling multiple exceptions

You can have multiple handlers:

Java

```
@ExceptionHandler(UserNotFoundException.class)
public ResponseEntity<?> handleUserNotFound(
    UserNotFoundException ex) {

    return ResponseEntity.status(404).body(ex.getMessage());
}
```

Java

```
@ExceptionHandler(IllegalArgumentException.class)
public ResponseEntity<?> handleIllegalArgument(
    IllegalArgumentException ex) {

    return ResponseEntity.badRequest().body(ex.getMessage());
}
```

Java

```
@ExceptionHandler(Exception.class)
public ResponseEntity<?> handleGenericException(
    Exception ex) {

    return ResponseEntity
        .status(500)
        .body("Something went wrong");
}
```

The last one is the **generic fallback**.

7. Returning a proper error response

In production applications, don't normally return just a `String`.

Create an error DTO:

Java

```
public record ErrorResponse(  
    int status,  
    String message,  
    String timestamp  
) {}
```

Then:

Java

```
@ExceptionHandler(UserNotFoundException.class)  
public ResponseEntity<ErrorResponse> handleUserNotFound(  
    UserNotFoundException ex) {  
  
    ErrorResponse error = new ErrorResponse(  
        404,  
        ex.getMessage(),  
        Instant.now().toString()  
    );  
  
    return ResponseEntity  
        .status(HttpStatus.NOT_FOUND)  
        .body(error);  
}
```

Response:

JSON

```
{  
  "status": 404,  
  "message": "User not found",  
  "timestamp": "2026-08-22T08:00:00Z"  
}
```

8. @ResponseStatus

Another approach:

Java

```
@ResponseStatus(HttpStatus.NOT_FOUND)  
public class UserNotFoundException extends RuntimeException {  
  
    public UserNotFoundException(String message) {
```

```
        super(message);  
    }  
}
```

Now throwing:

Java

```
throw new UserNotFoundException("User not found");
```

causes Spring to use `404`.

However, for a larger REST application, centralized handling with

`@RestControllerAdvice` generally gives you more control over the response structure.

9. `ResponseStatusException`

Spring also provides:

Java

```
throw new ResponseStatusException(  
    HttpStatus.NOT_FOUND,  
    "User not found"  
);
```

Useful for straightforward cases, but custom exceptions + centralized handling are often cleaner when you have a consistent API error contract.

10. Validation exceptions

Very common interview topic.

Suppose:

Java

```
@PostMapping("/users")  
public User createUser(  
    @Valid @RequestBody UserRequest request) {  
  
    ...  
}
```

And:

Java

```
public class UserRequest {  
  
    @NotBlank  
    private String name;  
  
    @Email  
    private String email;  
}
```

Invalid input can result in validation-related exceptions.

You can handle them centrally.

For example, depending on the Spring MVC/Spring Boot version and validation scenario, you'll commonly encounter:

```
MethodArgumentNotValidException  
HandlerMethodValidationException  
ConstraintViolationException
```

The exact exception depends on **how validation is being performed**.

11. @ControllerAdvice VS @RestControllerAdvice

Annotation	Purpose
@ControllerAdvice	Global advice for controllers
@RestControllerAdvice	@ControllerAdvice + @ResponseBody
@ExceptionHandler	Handles a particular exception
@ResponseStatus	Associates response status with exception/handler
@ResponseBody	Writes return value directly to HTTP response body

For REST APIs:

Java

```
@RestControllerAdvice
```

is usually the preferred choice.

12. Important distinction: stereotype annotations

Since you were confused about this earlier, remember this distinction:

Stereotype annotations

They identify a **role/component** managed by Spring:

```
@Component
@Service
@Repository
@Controller
@RestController
```

Conceptually:

"What kind of Spring component is this?"

Other annotations

They provide some other behavior/configuration:

```
@ControllerAdvice
@ExceptionHandler
@Autowired
@Bean
@Configuration
@Transactional
@RequestMapping
```

Conceptually:

"What should this component/method do?"

Don't worry too much about memorizing a rigid definition of "stereotype." The important interview distinction is **component role vs behavior/configuration**.

13. Best interview architecture

A typical production structure:

```
controller/
    UserController.java

service/
```

```
UserService.java

exception/
    UserNotFoundException.java
    GlobalExceptionHandler.java

dto/
    ErrorResponse.java
```

Flow:

```
UserController
    ↓
UserService
    ↓
UserNotFoundException
    ↓
@RestControllerAdvice
    ↓
@ExceptionHandler
    ↓
ErrorResponse
    ↓
HTTP 404
```

14. Interview questions you should be ready for

Basic

1. What is exception handling in Spring Boot?
2. What is `@ExceptionHandler` ?
3. What is `@ControllerAdvice` ?
4. Difference between `@ControllerAdvice` and `@RestControllerAdvice` .
5. How do you create a custom exception?

Intermediate

6. How does Spring find the correct exception handler?
7. What happens if both `ExceptionHandler.class` and `UserNotFoundException.class` handlers exist?
8. What happens if there is no exact exception handler?
9. How do you return a standardized error response?
10. How do you handle validation errors globally?

Advanced

11. What is Spring MVC's exception resolution mechanism?

12. What is `HandlerExceptionResolver`?
 13. How does `ExceptionHandlerExceptionResolver` work?
 14. Difference between `@ExceptionHandler`, `ResponseStatusException`, and `@ResponseStatus`.
 15. How would you design exception handling for a production microservice?
 16. How would you prevent sensitive exception details from being exposed to clients?
-

The 5 things I'd memorize for an interview

```
@ExceptionHandler
    ↓
Defines exception → handler method

@ControllerAdvice
    ↓
Makes exception handling applicable across controllers

@RestControllerAdvice
    ↓
@ControllerAdvice + @ResponseBody

Spring exception resolution
    ↓
Finds the most specific matching handler

No exact handler
    ↓
Can match superclass handler → generic Exception fallback
```

One interview-quality answer to memorize:

"In Spring Boot REST applications, we can use `@RestControllerAdvice` with `@ExceptionHandler` methods to centralize exception handling. When an exception propagates out of a controller, Spring MVC's exception-resolution mechanism looks for a matching handler. It selects the most specific applicable exception handler, and if an exact match isn't available, a superclass handler can handle the exception. The handler then returns a standardized HTTP response to the client."